Three-Tier Web Application on AWS

A hands-on AWS project demonstrating a **three-tier web architecture** with separate public, web, application, and database layers. An internet-facing Application Load Balancer (ALB) receives user traffic, private Nginx web servers serve the front end and route PHP requests, private PHP-FPM application servers process requests, and Amazon RDS for MySQL provides the database tier.

Project status: Built as a learning project. AWS resource names and availability can change as resources are started, stopped, or cleaned up.

Project Overview

The goal is to deploy a user-registration application on AWS using network segmentation, load balancing, Auto Scaling, and a managed relational database.

Application flow

1. A user opens the registration page in a browser over HTTP.
2. The internet-facing **Web ALB** receives the request on port 80.
3. The Web ALB forwards traffic to Nginx web servers in private subnets.
4. Nginx serves static content and forwards PHP requests to the internal **App ALB**.
5. The internal App ALB forwards requests to PHP-FPM application instances in private app subnets.
6. The application tier connects to Amazon RDS for MySQL on port 3306.
7. The registration flow displays a success page when the application completes the request.

Architecture

VPC: ThreeTierVpc --- 10.0.0.0/16
AWS Region shown in screenshots: US East (N. Virginia), us-east-1
Internet Gateway: ThreeTierGW
Route tables: ThreeTierPublicRouteTable and ThreeTierMainRouteTable

Layer Subnet CIDR block Purpose

Public PublicSubnet1 (AZ 10.0.0.0/20 Public entry and
a) internet-facing
load balancer

Public PublicSubnet2 (AZ 10.0.16.0/20 Second
b) Availability Zone
for the public
load balancer

Web WebSubnet1 (AZ a) 10.0.32.0/20 Private Nginx web
server

Web WebSubnet2 (AZ b) 10.0.48.0/20 Private Nginx web
server

App AppSubnet1 (AZ a) 10.0.64.0/20 Private PHP-FPM
application
server

App AppSubnet2 (AZ b) 10.0.80.0/20 Private PHP-FPM
application
server

Database DbSubnet1 (AZ a) 10.0.96.0/20 RDS DB subnet
group

Database DbSubnet2 (AZ b) 10.0.112.0/20 RDS DB subnet
group / Multi-AZ
placement

The remaining range 10.0.128.0/17 is reserved for future subnet expansion.

AWS Services and Technologies

- **Amazon VPC:** Custom network with public and private subnets.
- **Internet Gateway:** Internet connectivity for resources in public subnets.
- **Application Load Balancer:** Internet-facing Web ALB and internal App ALB.
- **Amazon EC2:** Nginx web instances and PHP application instances.
- **EC2 Auto Scaling Groups:** Web and app capacity managed through launch templates.
- **Nginx:** Serves static files and acts as a reverse proxy for PHP requests.

- **PHP and PHP-FPM:** Application runtime and FastCGI process manager.
- **Amazon RDS for MySQL:** Managed relational database with public access disabled.
- **Security Groups:** Intended to restrict traffic between tiers.
- **Availability Zones:** Resources are designed across two Availability Zones for improved resilience.

Traffic Flow

Internet / Browser

|

| HTTP : 80

v

Internet-facing Web ALB

|

| HTTP : 80

v

Private Web Tier (Nginx)

|

| PHP requests via HTTP : 80

v

Internal App ALB

|

| HTTP : 80

v

Private App Tier (PHP-FPM)

|

| MySQL : 3306

v

Amazon RDS for MySQL

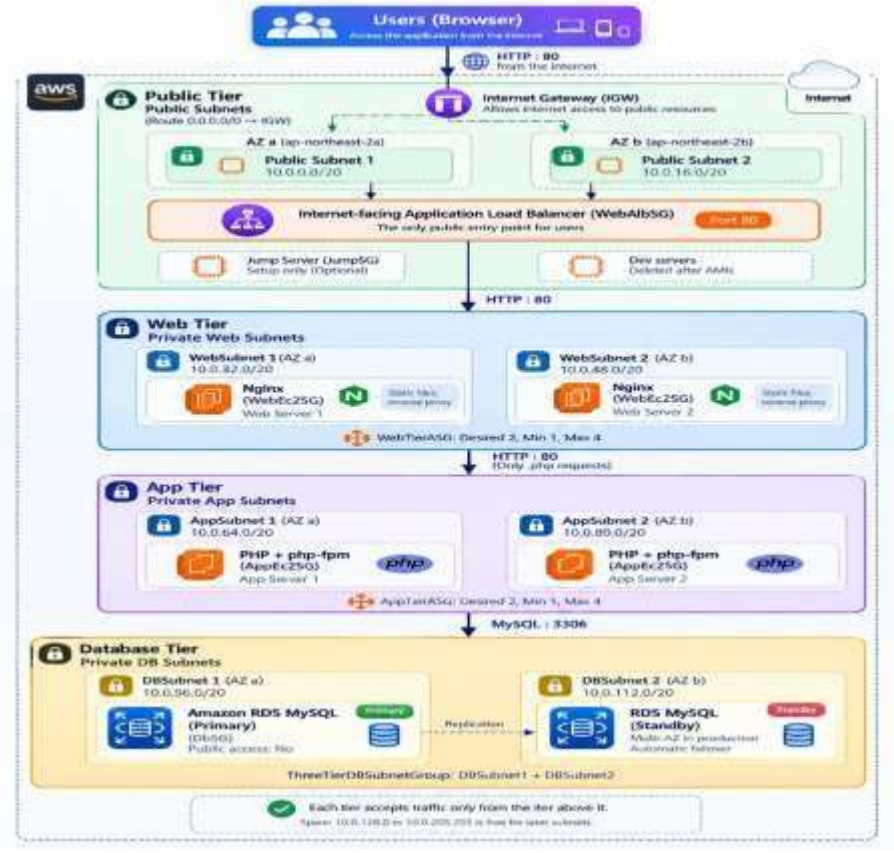
Security Design

Recommended security-group rules for this architecture:

- **Web ALB security group:** Allow inbound HTTP 80 from the internet; allow HTTPS 443 if TLS is configured.
- **Web EC2 security group:** Allow inbound HTTP 80 only from the Web ALB security group.
- **App ALB security group:** Allow inbound HTTP 80 only from the Web EC2 security group.
- **App EC2 security group:** Allow application traffic only from the App ALB security group.
- **RDS security group:** Allow MySQL 3306 only from the App EC2 security group.
- Keep application and database instances in private subnets. Avoid exposing SSH to 0.0.0.0/0; use a restricted administrative path such as a bastion host or Systems Manager where configured.

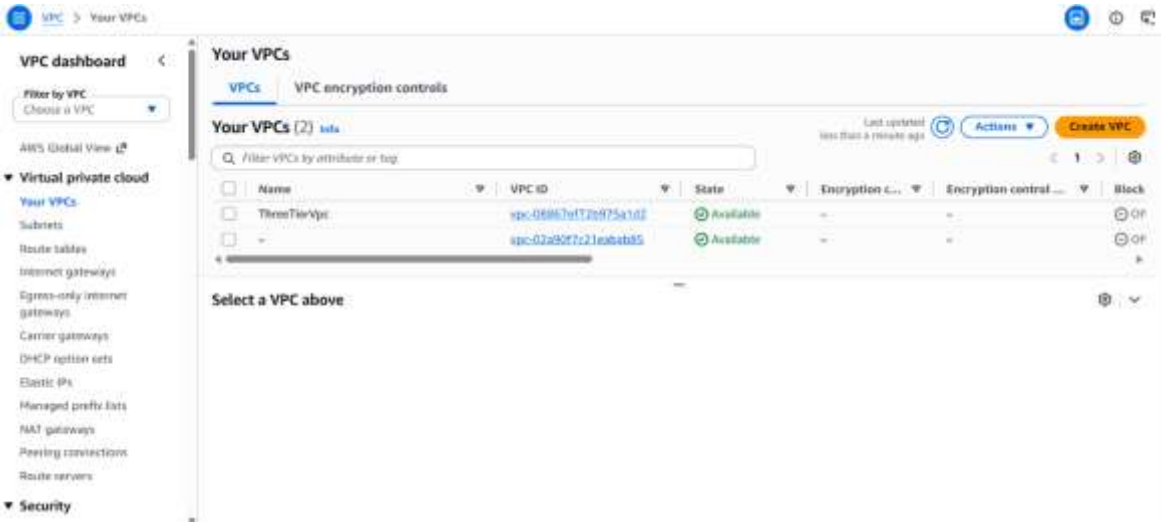
Verify the actual security-group rules before describing the deployment as fully secured. Screenshots of the architecture and resources alone do not prove every rule is configured exactly as recommended.

1. **Architecture Diagram**



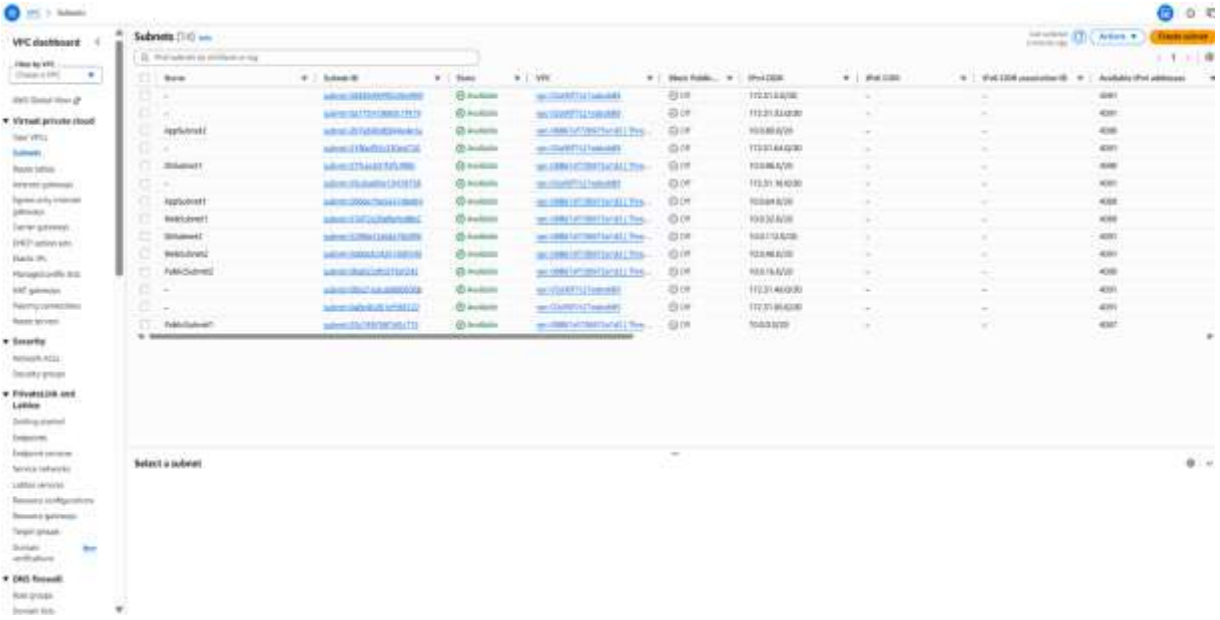
This diagram illustrates the complete three-tier web application architecture deployed on AWS. It shows the VPC, public and private subnets across two Availability Zones, internet-facing and internal Application Load Balancers, Nginx web servers, PHP-FPM application servers, and Amazon RDS for MySQL. It explains how traffic moves from the user's browser through the web and application tiers to the database tier.

2. VPC Overview



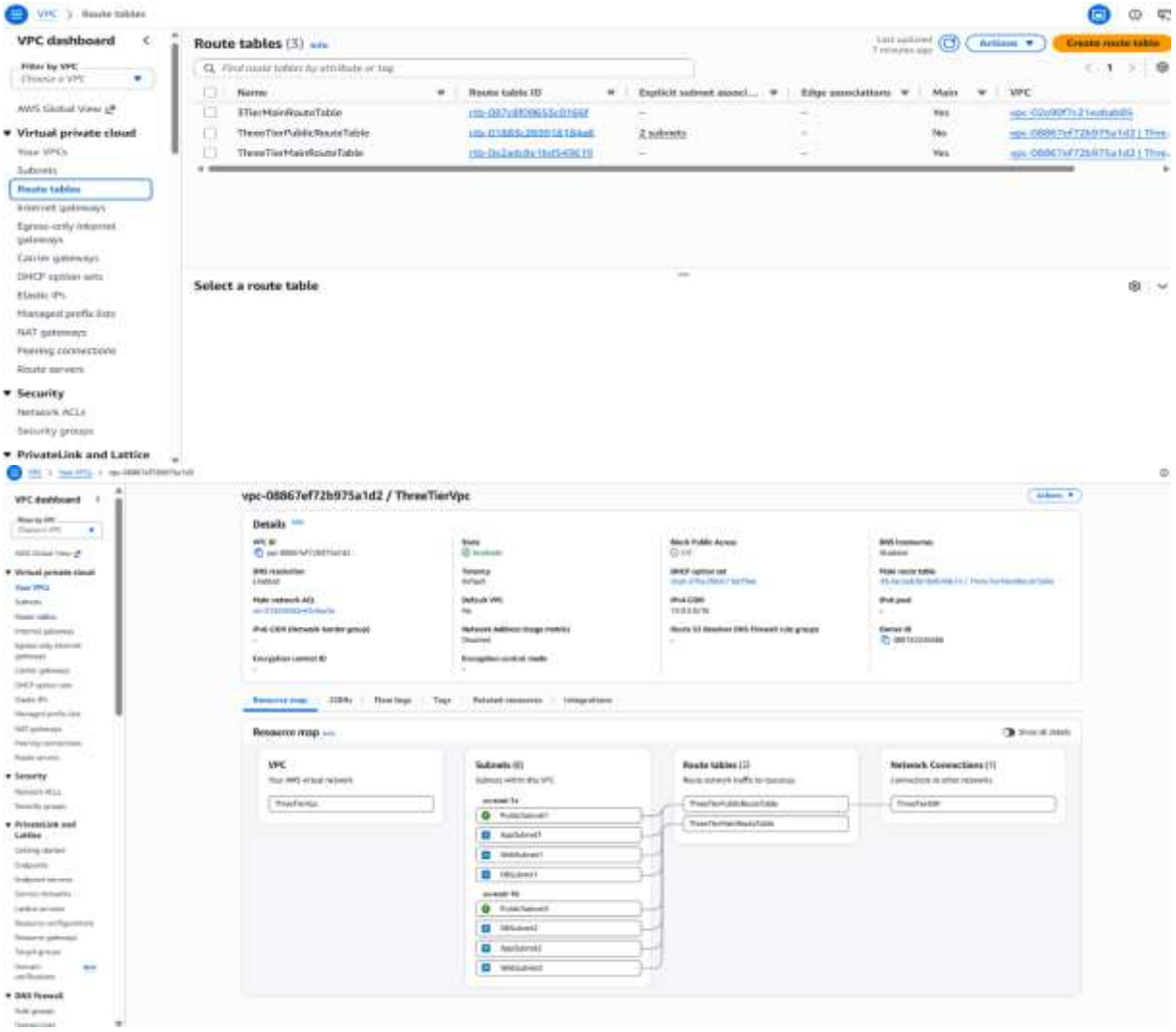
This screenshot shows the ThreeTierVpc virtual private cloud created in AWS with the IPv4 CIDR block 10.0.0.0/16. The VPC provides an isolated network environment for the project's AWS resources, including public subnets, private web subnets, application subnets, and database subnets.

3. Subnet Configuration



This screenshot displays the subnet configuration for the project VPC. It includes public, web, application, and database subnets distributed across two Availability Zones (us-east-1a and us-east-1b). Each tier uses a separate CIDR range to organize resources and support network isolation

4. Route Tables



This screenshot shows the route tables configured for the custom VPC, including ThreeTierPublicRouteTable and ThreeTierMainRouteTable. Route tables determine how network traffic is routed between subnets and network destinations. The public route table is intended to direct internet-bound traffic through the Internet Gateway, while private-tier routing is configured separately**.**

5. Application Load Balancers



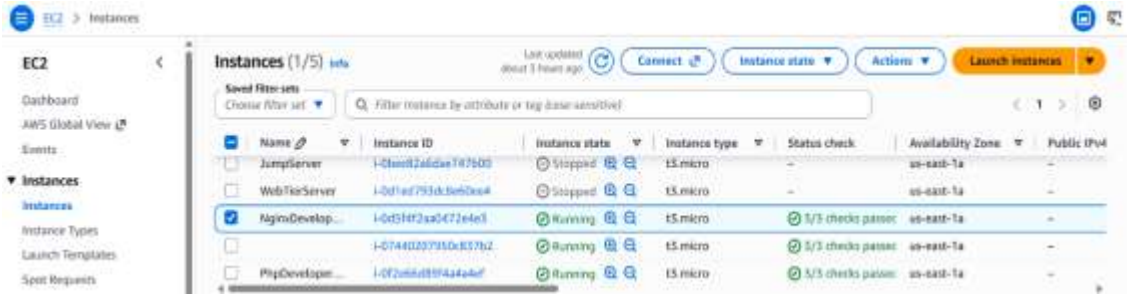
This screenshot displays the two Application Load Balancers created for the project: an internet-facing Web ALB and an internal App ALB. The Web ALB acts as the public entry point for incoming HTTP traffic, while the internal App ALB receives requests forwarded from the web tier and distributes them to application servers.

6. Auto Scaling Groups



This screenshot shows the WebtierAutoscallingGroup and ApptierAutoscallingGroup configured in AWS EC2 Auto Scaling. Both groups use launch templates and show a desired capacity of one instance, with a minimum of one and a maximum of four. Auto Scaling helps manage instance capacity according to the configured scaling policies.

7. EC2 Instances



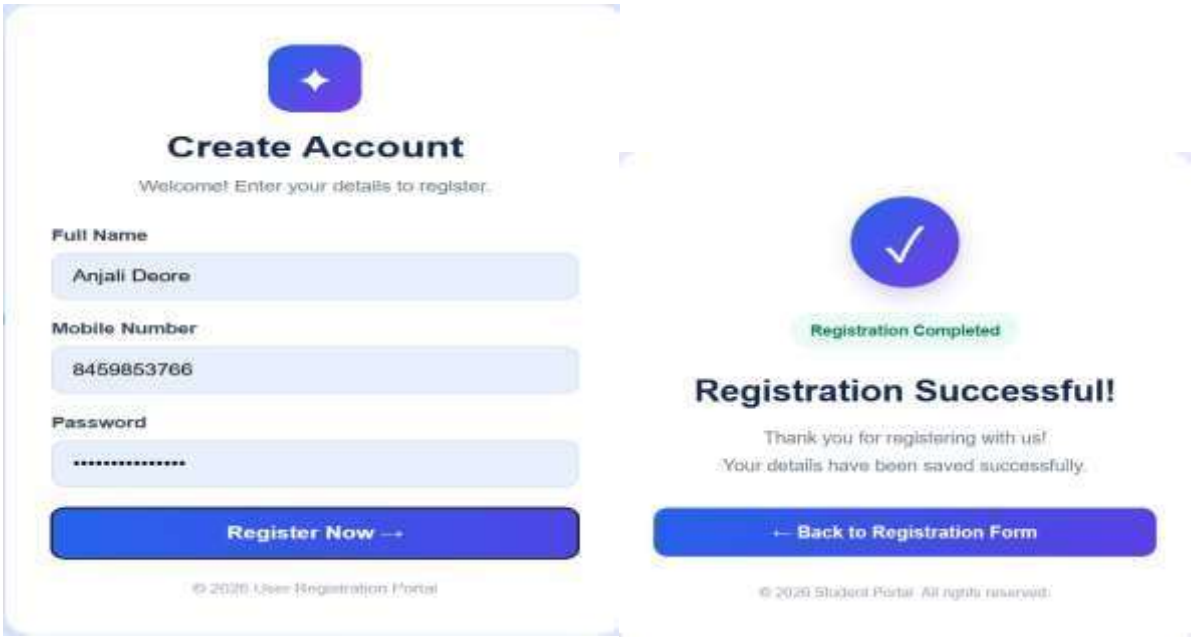
This screenshot displays the EC2 instances associated with the project, including the Jump Server, Web Tier Server, Nginx development server, and PHP development server. It shows the instance states, instance types, Availability Zones, and status checks. These instances support the project's web and application infrastructure, while the development instances can be used for testing and configuration.

8. Amazon RDS for MySQL



This screenshot shows the Amazon RDS database instance named database-1 in the us-east-1a Availability Zone. The database uses a MySQL-compatible engine and has an Available status, indicating that the database instance is running and ready for connections, subject to its network and security configuration. The database tier is designed to store application data separately from the web and application servers.

9. User Registration Form



This screenshot displays the front-end registration page for the web application. It includes fields for the user's full name, mobile number, and password, along with a Register Now button. The form provides the user interface for submitting registration details to the application's backend for processing and the registration success page displayed after the registration workflow completes. It presents a confirmation message and a button to return to the registration form. This demonstrates the intended user-facing success experience of the application; database persistence should be verified separately before claiming that submitted details were successfully stored in RDS.

What I Learned

- Designing a VPC with public and private subnets across Availability Zones.
- Understanding tier-to-tier traffic flow and least-privilege security-group rules.
- Configuring public and internal Application Load Balancers.
- Using EC2 launch templates and Auto Scaling Groups.
- Hosting a web front end with Nginx and processing PHP requests with PHP-FPM.
- Connecting an application tier to Amazon RDS for MySQL.

- Troubleshooting routing, load-balancer targets, web-server configuration, and application connectivity.

Future Improvements

- Configure HTTPS using AWS Certificate Manager and an ALB HTTPS listener.
- Validate health checks and target registration for both load balancers.
- Store database credentials securely in AWS Secrets Manager or Systems Manager Parameter Store.
- Use password_hash() and password_verify() for password handling; never store plaintext passwords.
- Add input validation, prepared SQL statements, and server-side error handling.
- Add CloudWatch monitoring, alarms, and application logs.
- Consider Systems Manager Session Manager for secure instance administration.